

Proyecto 3 Fundamentos de bases de datos

Juan Felipe León 202510397

David Hortua 202510382

1. Diseño de base de datos

a) Cuantificar entidades

Cientes	50.000
Hoteles	100
Reservas	250.000
Reseñas	250.000 en 2 años
Administradores	100
Votos utiles	365.000 en 2 años

- **Votos utiles:** $500 \text{ votos diarios} \times 730 \text{ días} = 365.000$

b) Operaciones de lectura y escritura

Entidad	Lecturas	Escrituras
Reseñas	Muy altas	Altas
Votos útiles	Altas	Medias
Respuestas admin	Medias	Medias
Hoteles	Altas	Bajas

- A partir del análisis realizado, se observa que la entidad Reseña concentra la mayor carga operativa del sistema, especialmente en operaciones de lectura, debido a la alta frecuencia de consultas realizadas por los usuarios. Asimismo, las operaciones de escritura también son significativas, ya que continuamente se crean nuevas reseñas, respuestas de administradores y votos de utilidad. Este comportamiento justifica el uso de MongoDB, dado que permite manejar

grandes volúmenes de información semiestructurada y consultas frecuentes de manera eficiente.

c) Cuantificaciones de operaciones

Operación	Cantidad diaria
Crear reseña	300–500
Consultar reseñas	10.000
Responder reseñas	200–300
Marcar útil	500
Consultas analíticas RFC	30

- Valores tomados del enunciado

Modelo conceptual:

- **Hotel**

Representa cada hotel perteneciente a la cadena Dann-Alpes. La información de los hoteles se encuentra almacenada en Oracle y es utilizada como referencia para asociar las reseñas realizadas por los clientes.

- **Cliente**

Representa los usuarios que realizan reservas en los hoteles de la cadena. Los clientes pueden crear reseñas sobre estadias completadas y votar reseñas como útiles.

- **Reserva**

Representa las reservas realizadas por los clientes. Cada reserva se asocia a un hotel y a un cliente. Una reserva únicamente puede

tener una reseña asociada y debe encontrarse en estado COMPLETADA para permitir la creación de una reseña.

- **Administrador**

Representa los administradores encargados de gestionar los hoteles de la cadena. Los administradores pueden responder reseñas, moderarlas y destacar comentarios relevantes.

- **Reseña**

Representa la opinión publicada por un cliente sobre una estadía realizada en un hotel. Cada reseña almacena una calificación, un comentario, información de utilidad, estado de publicación y la posible respuesta de un administrador.

- **VotoUtil**

Representa los votos realizados por usuarios autenticados para indicar que una reseña fue útil. Estos votos se almacenan asociados directamente a cada reseña.

- **RespuestaAdministrador**

Representa la respuesta oficial emitida por un administrador del hotel sobre una reseña publicada por un cliente.

Relaciones:

- **Relación 1 — Cliente y Reserva**

Un cliente puede realizar múltiples reservas, mientras que cada reserva pertenece a un único cliente.

Cardinalidad:

Cliente 1 ---- N Reserva

- **Relación 2 — Hotel y Reserva**

Un hotel puede tener múltiples reservas asociadas, mientras que cada reserva pertenece a un único hotel.

Cardinalidad:

Hotel 1 ---- N Reserva

- **Relación 3 — Reserva y Reseña**

Una reserva puede tener máximo una reseña asociada. Una reseña únicamente puede existir si está asociada a una reserva completada.

Cardinalidad:

Reserva 1 ---- 0..1 Reseña

- **Relación 4 — Cliente y Reseña**

Un cliente puede publicar múltiples reseñas a lo largo del tiempo, mientras que cada reseña pertenece a un único cliente.

Cardinalidad:

Cliente 1 ---- N Reseña

- **Relación 5 — Hotel y Reseña**

Un hotel puede tener múltiples reseñas asociadas, mientras que cada reseña corresponde a un único hotel.

Cardinalidad:

Hotel 1 ---- N Reseña

- **Relación 6 — Administrador y Reseña**

Un administrador puede responder múltiples reseñas, mientras que cada reseña tiene como máximo una respuesta oficial de administrador.

Cardinalidad:

Administrador 1 ---- N Reseña

- **Relación 7 — Reseña y VotoUtil**

Una reseña puede recibir múltiples votos de utilidad por parte de distintos usuarios autenticados.

Cardinalidad:

Reseña 1 ---- N VotoUtil

- **Relación 8 — Reseña y RespuestaAdministrador**

Una reseña puede tener como máximo una respuesta oficial realizada por un administrador.

Cardinalidad:

Reseña 1 ---- 0..1 RespuestaAdministrador

Decisión de diseño:

Relación	Tipo	Justificación
Hotel – Reseña	Referenciado	Hoteles existen en Oracle y muchas reseñas
Cliente – Reseña	Referenciado	Cliente pertenece a Oracle
Reserva – Reseña	Referenciado	Validación de negocio
Reseña – RespuestaAdmin	Embebido	Dependencia total

Reseña – VotoUtil	Embebido	Consulta conjunta frecuente
-------------------	----------	-----------------------------

Justificaciones:

1. La relación entre Hotel y Reseña se maneja mediante referencias utilizando el identificador del hotel. Esta decisión se toma debido a que la información principal de hoteles ya se encuentra almacenada en Oracle y porque un hotel puede llegar a tener una gran cantidad de reseñas a lo largo del tiempo. Embebir todas las reseñas dentro de un único documento de hotel produciría documentos excesivamente grandes y afectaría el rendimiento de actualización y consulta.
2. La relación entre Cliente y Reseña se implementa mediante referencias usando el identificador del cliente. Esto se debe a que la información de clientes pertenece al sistema relacional existente en Oracle y las reseñas únicamente requieren conservar el identificador del usuario que las realizó.
3. La relación entre Reserva y Reseña se maneja mediante referencias utilizando el identificador de la reserva. Esto permite validar las reglas de negocio relacionadas con reservas completadas y evitar duplicación innecesaria de información proveniente del sistema transaccional relacional.
4. La respuesta del administrador se almacena embebida dentro del documento de reseña, debido a que depende completamente de la existencia de la reseña y normalmente se consulta junto con ella. Además, la cardinalidad es uno a uno y el tamaño de la información es reducido.
5. Los votos de utilidad se almacenan embebidos dentro de cada reseña, ya que representan información dependiente de esta y son consultados frecuentemente junto con el contenido de la reseña. El tamaño esperado de estos documentos es manejable y el acceso conjunto mejora el rendimiento de lectura.

d) Descripción grafica

Ejemplo de JSON:

```
{
  "_id": ObjectId("1434233242313134"),
  "reseña_id": 1,
  "hotel_id": 10,
  "cliente_id": 25,
  "reserva_id": 120,
  "calificacion": 5,
  "comentario": "Excelente atención y muy buenas instalaciones",
  "fecha": ISODate("2026-05-20T10:30:00Z"),
  "estado": "PUBLICADA",
  "utilidad": 14,
  "destacada": false,
  "respuesta_admin": {
    "admin_id": 3,
    "texto": "Gracias por hospedarse con nosotros",
    "fecha": ISODate("2026-05-21T09:00:00Z")
  },
  "votos utiles": [
    {
      "usuario_id": 40,
      "fecha": ISODate("2026-05-21T12:00:00Z")
    },
    {
      "usuario_id": 41,
      "fecha": ISODate("2026-05-21T13:00:00Z")
    }
  ]
}
```

La respuesta del administrador se almacena embebida dentro de la reseña debido a su dependencia directa y a que normalmente se consulta junto con la reseña.

Los votos útiles se almacenan embebidos dentro de la reseña para facilitar el acceso conjunto y evitar consultas adicionales sobre información altamente relacionada.

Los identificadores de hotel, cliente y reserva se almacenan mediante referencias, ya que estas entidades pertenecen al sistema relacional implementado en Oracle y únicamente se requiere conservar la asociación lógica con la reseña.

Script de creación y validación:

```
db.createCollection("reseñas", {
  validator: {
    $jsonSchema: {
      bsonType: "object",

      required: [
        "reseña_id",
        "hotel_id",
        "cliente_id",
        "reserva_id",
        "calificacion",
        "comentario",
        "fecha",
        "estado"
      ],

      properties: {

        reseña_id: {
          bsonType: "int",
          description: "Identificador único de la reseña"
        },

        hotel_id: {
          bsonType: "int",
          description: "Identificador del hotel"
        },

        cliente_id: {
          bsonType: "int",
          description: "Identificador del cliente"
        },

        reserva_id: {
```



```
    bsonType: "int",
    description: "Identificador de la reserva"
  },

  calificacion: {
    bsonType: "int",
    minimum: 1,
    maximum: 5,
    description: "Calificación de la reseña"
  },

  comentario: {
    bsonType: "string",
    description: "Comentario escrito por el cliente"
  },

  fecha: {
    bsonType: "date",
    description: "Fecha de creación de la reseña"
  },

  estado: {
    enum: ["PUBLICADA", "ELIMINADA"],
    description: "Estado actual de la reseña"
  },

  utilidad: {
    bsonType: "int",
    minimum: 0,
    description: "Cantidad de votos útiles"
  },

  destacada: {
    bsonType: "bool",
    description: "Indica si la reseña está destacada"
  },

  respuesta_admin: {
    bsonType: "object",
    properties: {

      admin_id: {
        bsonType: "int"
      },

      texto: {
        bsonType: "string"
      },

      fecha: {
        bsonType: "date"
      }
    }
  },
}
```

```
votos_utiles: {  
  bsonType: "array",  
  
  items: {  
    bsonType: "object",  
  
    properties: {  
  
      usuario_id: {  
        bsonType: "int"  
      },  
  
      fecha: {  
        bsonType: "date"  
      }  
    }  
  }  
}  
}  
}  
}  
}  
}  
}  
})
```

```
items: {  
  bsonType: "object",  
  
  properties: {  
  
    usuario_id: {  
      bsonType: "int"  
    },  
  
    fecha: {  
      bsonType: "date"  
    }  
  }  
}  
}  
}  
}  
}  
}  
}  
})  
< { ok: 1 }
```

Insert valido:

```
db.reseñas.insertOne({

  reseña_id: 1,

  hotel_id: 10,

  cliente_id: 25,

  reserva_id: 120,

  calificacion: 5,

  comentario: "Excelente atención",

  fecha: new Date(),

  estado: "PUBLICADA",

  utilidad: 12,

  destacada: false,

  respuesta_admin: {
    admin_id: 3,
    texto: "Gracias por hospedarse",
    fecha: new Date()
  },

  votos utiles: [
    {
      usuario_id: 40,
      fecha: new Date()
    }
  ]
})
```

```

    respuesta_admin: {
      admin_id: 3,
      texto: "Gracias por hospedarse",
      fecha: new Date()
    },

    votos utiles: [
      {
        usuario_id: 40,
        fecha: new Date()
      }
    ]
  })
}
{
  acknowledged: true,
  insertedId: ObjectId('6a12153f9951ebc37101ee8c')
}

```

Insert invalido:

```

db.reseñas.insertOne({

  reseña_id: 2,

  hotel_id: 11,

  cliente_id: 30,

  reserva_id: 150,

  calificacion: 10,

  comentario: 12345,

  estado: "ACTIVA"
})

```

```

> db.reseñas.insertOne({

  reseña_id: 2,

  hotel_id: 11,

  cliente_id: 30,

  reserva_id: 150,

  calificacion: 10,

  comentario: 12345,

  estado: "ACTIVA"
})
MongoServerError: Document failed validation

```

2. Diseño de Sentencias MongoDB

RFC1 – Top 10 hoteles por calificación promedio en un período

a) Documentación

Colección utilizada: **resenias**

Atributos utilizados:

- **hotel_id** (int): identificador del hotel, usado para agrupar los resultados
- **calificacion** (int, 1–5): calificación dada por el cliente, sobre la cual se calcula el promedio
- **fecha** (date): fecha de creación de la reseña, usada para filtrar por período
- **estado** (string): permite filtrar solo reseñas con estado "PUBLICADA"

Tipo de agregación y justificación:

Se utilizó el pipeline **\$match** → **\$group** → **\$sort** → **\$limit** → **\$project**.

- **\$match** filtra primero por período (año 2024) y estado **PUBLICADA**, aprovechando el índice compuesto (**hotel_id**, **calificacion**, **fecha**) para una lectura eficiente.
- **\$group** reduce todas las reseñas de cada hotel a un único documento con el promedio de calificación (**\$avg**) y el conteo total (**\$sum: 1**). Esta es la operación central de la consulta.
- **\$sort** ordena descendientemente por **calificacion_promedio** para obtener los mejores primero.
- **\$limit 10** garantiza devolver únicamente el top requerido.
- **\$project** formatea la salida eliminando el **_id** interno y renombrando campos.

Se escogió **\$group** con **\$avg** porque permite calcular el promedio en una sola pasada sobre los documentos filtrados, sin necesidad de consultas adicionales ni joins entre colecciones.

b) Sentencia ejecutada

```
db.resenias.aggregate([
  {$match:{
    estado:"PUBLICADA",
    fecha:{$gte:ISODate("2024-01-01"),$lte:ISODate("2024-12-31")}}
  },
  {$group:{
    _id:"$hotel_id",
    calificacion_promedio:{$avg:"$calificacion"},
    total_resenas:{$sum:1}
  }},
  {$sort:{calificacion_promedio:-1}},
])
```

```
{ $limit: 10 },
{ $project: {
  _id: 0,
  hotel_id: "$_id",
  calificacion_promedio: { $round: [ "$calificacion_promedio", 2 ] },
  total_resenas: 1
} }
})
```

Resultados obtenidos:

hotel_id	calificacion_promedio	total_resenas
5	3.5	34
7	3.49	37
6	3.44	34
2	2.98	50
4	2.95	37
3	2.88	56
1	2.85	52

RFC2 – Evolución de la reputación de un hotel mes a mes

a) Documentación

Colección utilizada: **resenias**

Atributos utilizados:

- **hotel_id** (int): identificador del hotel a analizar, usado en el filtro inicial
- **calificacion** (int, 1–5): valor sobre el cual se calcula el promedio mensual
- **fecha** (date): de esta fecha se extraen el año y el mes usando **\$year** y **\$month**
- **estado** (string): filtra solo reseñas "PUBLICADA"

Tipo de agregación y justificación:

Se utilizó el pipeline **\$match** → **\$group** → **\$sort** → **\$project**.

- **\$match** filtra por **hotel_id**, año (2024) y estado **PUBLICADA**.
- **\$group** agrupa por mes usando los operadores de fecha **\$month** sobre el campo **fecha**, calculando el promedio de calificación y el total de reseñas por

mes. Se escogió este enfoque porque permite extraer el período directamente en el pipeline sin necesidad de transformar el dato antes de almacenarlo.

- **\$sort** ordena cronológicamente por mes para facilitar la visualización en gráfico de línea.
- **\$project** formatea la salida con el número de mes como campo principal.

Se escogió **\$group** con **\$month** porque MongoDB permite operar sobre fechas directamente en el pipeline, lo que evita redundancia en el modelo de datos y mantiene el campo **fecha** como único campo temporal en el documento.

b) Sentencia ejecutada

```
db.resenias.aggregate([
  {$match:{
    hotel_id:1,
    estado:"PUBLICADA",
    fecha:{$gte:ISODate("2024-01-01"),$lte:ISODate("2024-12-31")}
  }},
  {$group:{
    _id:{mes:{$month:"$fecha"}},
    calificacion_promedio:{$avg:"$calificacion"},
    total_resenas:{$sum:1}
  }},
  {$sort:{"_id.mes":1}},
  {$project:{
    _id:0,
    mes:"$_id.mes",
    calificacion_promedio:{$round:["$calificacion_promedio",2]},
    total_resenas:1
  }}
])
```

Resultados obtenidos (Hotel 1, año 2024):

mes	calificacion_promedio	total_resenas
1	3.00	3
2	3.00	3
3	2.50	2
4	3.20	5
5	2.67	3
6	3.00	5
7	2.5	4

8	2.4	5
9	2.5	10
10	2.6	5
11	4	1
12	3.67	6

RFC3 – Perfil comparativo de hoteles por ciudad

a) Documentación

Colección utilizada: **resenias**

Atributos utilizados:

- **hotel_id** (int): identificador del hotel, usado para agrupar y filtrar por ciudad (los hotel_ids de cada ciudad se obtienen de Oracle)
- **calificacion** (int, 1–5): para calcular el promedio general del hotel
- **estado** (string): filtra solo reseñas "PUBLICADA"
- **respuesta_admin** (object o null): su existencia (\$ifNull) permite contar reseñas con respuesta oficial
- **destacada** (bool): permite contar reseñas destacadas por hotel

Tipo de agregación y justificación:

Se utilizó el pipeline **\$match** → **\$group** → **\$addFields** → **\$sort** → **\$project**.

- **\$match** filtra los hoteles de la ciudad seleccionada (pasados como array de IDs) y solo reseñas **PUBLICADA**.
- **\$group** calcula por hotel: promedio de calificación, total de reseñas, cantidad con respuesta de admin (usando **\$cond** + **\$ifNull** para detectar si el campo existe) y cantidad destacadas.
- **\$addFields** calcula los porcentajes: **pct_con_respuesta** y **pct_destacadas** usando **\$divide** y **\$multiply**.
- **\$sort** ordena de mayor a menor calificación promedio.

Se escogió este enfoque porque **\$group** con **\$cond** permite calcular múltiples métricas en una sola pasada, evitando consultas adicionales. El campo **bajo_promedio_ciudad** se calcula en la aplicación comparando cada hotel contra el promedio general.

La ciudad no se almacena en MongoDB (vive en Oracle), por lo que los **hotel_ids** se pasan como parámetro desde la aplicación APEX tras una consulta previa a Oracle.

b) Sentencia ejecutada

```
db.resenias.aggregate([
```



```

{$match:{hotel_id:{$in:[1,2,3]},estado:"PUBLICADA"}},
{$group:{
  _id:"$hotel_id",
  calificacion_promedio:{$avg:"$calificacion"},
  total_resenas:{$sum:1},
  con_respuesta:{$sum:{$cond:[$ifNull["$respuesta_admin",false]],1,0]}},
  destacadas:{$sum:{$cond:[$destacada],1,0}}}
}},
{$addFields:{
  calificacion_promedio:{$round:[$calificacion_promedio,2]},

pct_con_respuesta:{$round:[$multiply:[$divide:[$con_respuesta,$total_resenas]],100]},1}},

pct_destacadas:{$round:[$multiply:[$divide:[$destacadas,$total_resenas]],100]},1}
}},
{$sort:{calificacion_promedio:-1}},
{$project:{
  _id:0,
  hotel_id:"$_id",
  calificacion_promedio:1,
  total_resenas:1,
  pct_con_respuesta:1,
  pct_destacadas:1
}}
])

```

Resultados obtenidos (Ciudad: Bogotá, hotel_ids: 1, 2, 3):

hotel_id	calificacion_promedio	total_resenas	pct_con_respuesta	pct_destacadas
3	3.04	80	40.0%	1.2%
2	2.93	74	41.9%	1.4%
1	2.76	76	40.8%	1.3%

Promedio ciudad (Bogotá): ~2.91

- Hotel 3:  sobre el promedio
- Hotel 2:  sobre el promedio
- Hotel 1:  bajo el promedio

3 Disclaimer falta de requerimientos

Intentamos conectar APEX con MongoDB mediante una API REST desarrollada en FastAPI y desplegada en Render. La API funciona correctamente al consultarla directamente desde el navegador, pero al llamarla desde APEX el navegador la bloquea por una política de seguridad llamada CORS. Intentamos solucionarlo de varias formas sin éxito, ya que el problema está en la configuración del servidor de Oracle APEX de la universidad, a la cual no tenemos acceso. Resolver esto requeriría permisos de administrador sobre el servidor, lo que está fuera de nuestro alcance. Como evidencia adjuntamos en el repo de Github el código fuente completo de la API con todos los endpoints implementados y capturas de su correcto funcionamiento.